Adding Subtasks to Task Manager

https://gemini.google.com/app/1e2f9404c5488862

User prompt: task_manager.py: ```python from datetime import date def add_task(tasks, title, due_date=None, priority="medium"): """Add a new task to the task list. Args: tasks (list): The list of existing task dictionaries. title (str): The task's title. due_date (date, optional): The task's due date. Defaults to None. priority (str): One of "high", "medium", "low". Defaults to "medium". Returns: list: The updated list of tasks. """ tasks.append({"title": title, "due_date": due_date, "priority": priority, "completed": False}) return tasks def mark_complete(tasks, title): """Mark a task as completed by title. Args: tasks (list): The list of task dictionaries. title (str): The title of the task to mark complete. Returns: bool: True if a matching task was found and updated, False otherwise. """ for task in tasks: if task["title"] == title: task["completed"] = True return True return False def get_overdue_tasks(tasks, current_date): overdue = [] for task in tasks: if task["due_date"] < current_date and not task["completed"]: overdue.append(task) return overdue def calculate_completion_rate(tasks): if not tasks: return 0.0 completed = sum(1 for t in tasks if t["completed"]) return completed / len(tasks) ``` Write a function add_subtask(tasks, parent_title, subtask_title) that adds a subtask to an existing task.

Response: Here is the implementation of `add_subtask`, along with a slight adjustment to `add_task` so that newly created tasks automatically include an empty list for subtasks.

```
def add_subtask(tasks, parent_title, subtask_title):
    """Add a subtask to an existing parent task by title.

    Args:
        tasks (list): The list of task dictionaries.
        parent_title (str): The title of the parent task.
        subtask_title (str): The title of the subtask to add.

    Returns:
        bool: True if the parent task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == parent_title:
            # Ensure the subtasks list exists before appending
            if "subtasks" not in task:
                task["subtasks"] = []

            task["subtasks"].append({
                "title": subtask_title,
                "completed": False
            })
            return True
    return False
```

Suggested Update to `add_task`

To keep the data structure consistent across all tasks, update `add_task` to initialize the `subtasks` field when creating a new task:

```
def add_task(tasks, title, due_date=None, priority="medium"):
    tasks.append({
        "title": title,
```

```
        "due_date": due_date,
        "priority": priority,
        "completed": False,
        "subtasks": []
    })
    return tasks
```
